We thank the reviewers for their time and suggestions. This rebuttal addresses the concerns, and points to the modifications (in blue) in the revised manuscript.

— Review_1425A —

A–Q1 **Listing improperly floated.** Thanks for noticing. We have fixed this issue.

A–Q2 **Figures 6 and 9** We have added an introduction to these figures and re-worked the text.

A–Q3 **Simpoint methodology** They are not independent simpoint samples, but different inputs for the same benchmark (some SPEC 2017 benchmarks include multiple inputs that lead to different program behaviour). All SimPoint intervals for each benchmark-input pair are aggregated. We have clarified this in Section IV-A.

— Review_1425B —

B–Q1 **Only improves performance in about half of the benchmarks** The apparent regressions in Figure 4 are due to the hatched bars being visually misleading. In actuality, all but two of the workloads have a performance regression, both of only 0.3%. All other workloads improve or maintain IPC. We have annotated all bars regardless of % change to make this clear.

B–Q2 **Why does saving on ports help in constrained processors?**

B–Q3 **Re-routing memory instructions & hardware cost of PG-MDP** PG-MDP requires no redirection of u-ops in the LSU. Labelled loads simply carry an additional bit, available since decode, which can hardwire to MDP to always return no dependency on lookup. The additional decode overhead is likewise trivial for AArch64 and RISCV (X86 is discussed in response to reviewer C), as the opcode hint is in a fixed position in the ISA and requires no additional critical path logic beyond what is already required to decode regular load instructions. This brings the total hardware overhead for PG-MDP for these architectures to a handful of additional logic gates.

B–Q4

— Review_1425C —

Profiling and evaluating SPEC. Profiling was carried on using the training inputs of the Spec suite. Applications with several training inputs used all of them for training. For evaluation, the reference inputs were used (See Section III-B and III-C).

MDP used for experiments. Yes, the MDP used in our experiments is XiangShan's version of Store Sets. The SSIT remains direct-access, so entries are overwritten on collision. The LFST supports multiple stores per set; on allocation, an invalid slot is selected if available, otherwise a victim is selected using round-robin replacement.

x86. To complete! Need clarification probably in the text.

When are MDP misprediction is flushed. Upon detection, when a store discovers that a younger load has executed prematurely and violated memory ordering.

— Review_1425D —

Comparison to other approaches. TODO.

— Review_1425E —

Complexity. TODO.

PG-MDP: Profile-Guided Memory Dependence Prediction for Area-Constrained Cores

HPCA 2027 Submission **#1425**
Confidential Draft
Do NOT Distribute!!

***Abstract*—Memory Dependence Prediction (MDP) is a speculative technique to predict which stores, if any, a given load will depend on. Area-constrained cores are increasingly relevant in various applications such as energy-efficient or edge systems, and often have limited space for MDP tables. This leads to a high rate of false dependencies as memory independent loads alias with unrelated predictor entries, causing unnecessary stalls in the processor pipeline.**

The conventional way to address this problem is with greater predictor size or complexity, but this is unattractive on area-constrained cores. This paper demonstrates that targeting the predictor working set delivers the majority of available performance without scaling any hardware structures. We achieve this with profile-guided memory dependence prediction (PG-MDP), a hardware-software co-design to label consistently memory independent loads via their opcode and remove them from the MDP working set. These loads bypass querying the MDP and always issue as soon as possible. In the event that a labeled load incorrectly passes a store to the same address, a rollback is triggered as usual but no new MDP entry is created. Across the SPECspeed 2017 suites, PG-MDP reduces MDP load queries by 80%, false dependencies by 84%, and improves geomean IPC for a small (ROB=128) simulated core by 4.6% (to within 1.2% of the IPC when using 8x more predictor entries), with zero storage cost and no additional instruction bandwidth.

I. Introduction

Memory dependence prediction (MDP) is a speculative technique used in out-of-order processors to increase available instruction-level parallelism (ILP) by predicting the stores on which a given load instruction will depend, as well as identifying loads which are memory independent. As recent work [**?**] exploring MDP designs in modern commercial processors reveals, the storage budget allocated to these memory dependence predictors is extremely small, with results suggesting even high-performance cores may not break the 1KB barrier of storage, and area-constrained efficiency-focused cores could be using as little as 50-100 bytes.

Recent work [**?**], [**?**], [**?**] exploring MDP designs in modern commercial processors finds that the storage budget allocated to memory dependence predictors is extremely small, with results suggesting even high-performance cores may not break 1KB of storage, and area-constrained efficiency-focused cores could be using as little as 50-100 bytes. For example, the Apple M-series efficiency cores measured in [**?**] are listed as having 21 fully-associative entries, with each entry containing 12 load tag bits, 12 store tag bits, 3 counter bits and 5 LRU bits, coming to just 84 bytes. This is unsurprising when considering the design constraints of these predictors; because

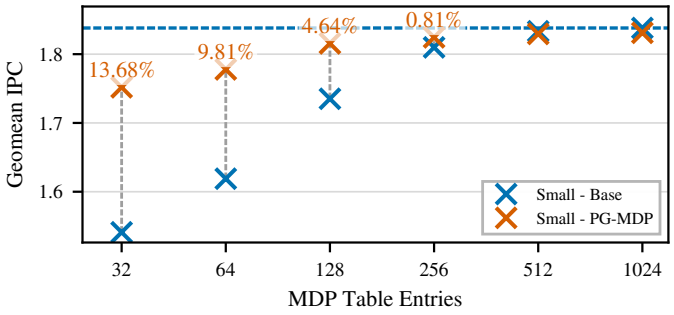


Fig. 1: Base and improved IPC across SPECspeed 2017 for increasing XS Store Sets [**?**] sizes on a small core (ROB=128). With PG-MDP, 128 entries deliver within 1.2% of available IPC from 1024 entries.

MDPs are typically queried by both loads and stores, their tables must be highly multi-ported to ensure the pipeline is never blocked by memory operations waiting to receive predictions. For example, the open-source XiangShan processor [**?**] implements a Store Sets-based [**?**] predictor with 8 read ports. This places a high premium on area allocated to memory dependence predictors, which is further exacerbated in area-constrained cores where additional area is highly contested by performance-critical components (e.g. branch prediction), and low power usage is a top priority. As such, these smaller cores employ smaller predictors and struggle with a high rate of false positives (false dependencies) due to hash collisions/aliasing, causing loads to wait for preceding stores that are incorrectly predicted to be dependent.

The importance of such area-constrained cores has also grown over time. Modern systems increasingly deploy heterogeneous processors containing both performance and efficiency-focused core designs, combining larger high-performance cores with small energy-efficient (but still out-of-order) cores, as seen in ARM's big.LITTLE systems and both Apple's & Intel's performance and efficiency cores. These efficiency cores share die area with high-performance cores under strict budgets for both area and power. This pattern is no longer confined to mobile devices, and this heterogeneous design can now be found across mobile, desktop, and server designs. Even as processors in high-end mobile devices scale to rival those found in laptops, they are often paired with efficiency cores to maximize battery life. Area-constrained

cores are further found as standalone cores in edge systems, or efficiency tiles in chiplet architectures. In all these cases, out-of-order execution components are scaled to fit the given area envelope rather than to maximize performance. As these area-constrained cores are increasingly tasked with demanding general-purpose computation while striving to consume as little energy as possible, increasing ILP without incurring additional area or power costs becomes more pressing.

One way to increase available ILP is to reduce false dependencies in MDP. This can be addressed by increasing predictor capacity to reduce aliasing and improve accuracy, but as established this is either unattractive or simply infeasible. Instead, this paper improves accuracy by reducing the **predictor working set**. We define the predictor working set as the set of instructions that actively index into the predictor during a given execution window. A typical MDP working set is every load and store instruction, as seen in Store Sets [**?**] and many of the predictors listed in [**?**]. There exist several sophisticated hardware techniques which can at least partially reduce the MDP working set, such as load elimination [**?**] or value prediction [**?**], but each of these also requires space for large, power-hungry predictors, as well as abundant ILP to return the most benefit, which are precisely the resources an area-constrained core lacks. Prior work demonstrates that a considerable portion of load instructions in general-purpose workloads rarely or never exhibit in-flight dependencies even across varying inputs [**?**], [**?**], [**?**], and so these loads are ideal candidates to be removed from the MDP working set via hardware-software co-design in the ISA without incurring area or power cost beyond trivial additional decode logic.

We achieve this with **profile-guided memory dependence prediction (PG-MDP)**, a hardware-software co-design which identifies consistently memory-independent loads through profiling and labels them via an alternate opcode, allowing them to bypass MDP load queries and issue as soon as their register dependencies are satisfied. Labeled loads still trigger the normal squash and recovery mechanism upon a memory order violation to preserve correctness, but do not update the predictor. Using a modern variant of the Store Set predictor [**?**] found in the XiangShan core [**?**] (referred to as XS Store Sets), we show that for an appropriate working set even a small predictor is able to deliver performance competitive with a very large predictor. PG-MDP reduces average dynamic MDP load queries by 80% and false dependencies by 84% across SPECspeed 2017, providing a 4.6% geomean IPC gain on a small (ROB = 128) simulated core, and within 1.2% IPC of using an MDP with 8x more entries, as shown in Figure 1.

A. Contributions

This paper makes the following contributions:

- **Reframes MDP Aliasing:** We show that the dominant source of false dependencies in memory dependence prediction does not stem solely from insufficient predictor capacity, but equally from an unnecessarily large predictor working set that captures both memory dependent and independent loads. We demonstrate that framing MDP aliasing as only a capacity problem is misleading, and that shrinking the working set is as effective as growing the predictor.
- **Introduces PG-MDP:** By leveraging profile-guided memory dependence prediction, we reduce dynamic MDP load queries by 80% on average across SPECspeed 2017, and false dependencies by 84%.
- **Delivers Zero-Storage IPC Gains:** Using gem5 [**?**] to simulate a core of comparable size to efficiency-focused processors found in production today, we demonstrate a 4.6% IPC gain evaluated against a 300B (3x expected real-world budget) XiangShan-variant Store Sets predictor (referred to as XS Store Sets) [**?**], without incurring any additional predictor capacity or power costs.

II. Background

This section outlines the fundamental concepts in scheduling memory operations in out-of-order execution. It then introduces Store Sets as a foundational memory dependence predictor algorithm as well as the modern variation found in the XiangShan processor.

A. Loads in Out-of-Order Execution

A key structure in out-of-order execution is the load-store queue (LSQ), used to hold in-flight memory operations and perform memory disambiguation. When load instructions are dispatched, they are inserted into the load queue (LQ), and query the memory dependence predictor (MDP) for a predicted store(s) to wait on before execution. Once the source operands are ready and all predicted dependent stores have executed, loads search backwards through the store queue (SQ) for store-forwarding opportunities. If no matching addresses are found, loads obtain their data from the cache.

When a load searches the SQ, not all earlier stores may have resolved their addresses. As the majority of the time a store's address will not match with the load's, it is often profitable to speculatively ignore these stores and issue the load as soon as possible. When the unresolved store eventually computes its address, it searches forward through the LQ to find loads with matching addresses which have already executed. If an address overlap is found, the load has received stale data and a memory order violation has occurred. The processor must flush all in-flight instructions from the load onwards and re-fetch.

The goal of the MDP is to minimize violation events while maximizing available ILP, allowing memory independent loads to issue as soon as possible and dependent loads to wait only for the necessary stores. When a load is incorrectly stalled on a store that does not resolve to the same address this is called a false dependency. Simple MDP algorithms typically prioritize reducing the rate of violations over the rate of false dependencies.

B. Store Sets

A seminal paper in memory dependence prediction is 'Memory Dependence Prediction using Store Sets' [**?**]. This

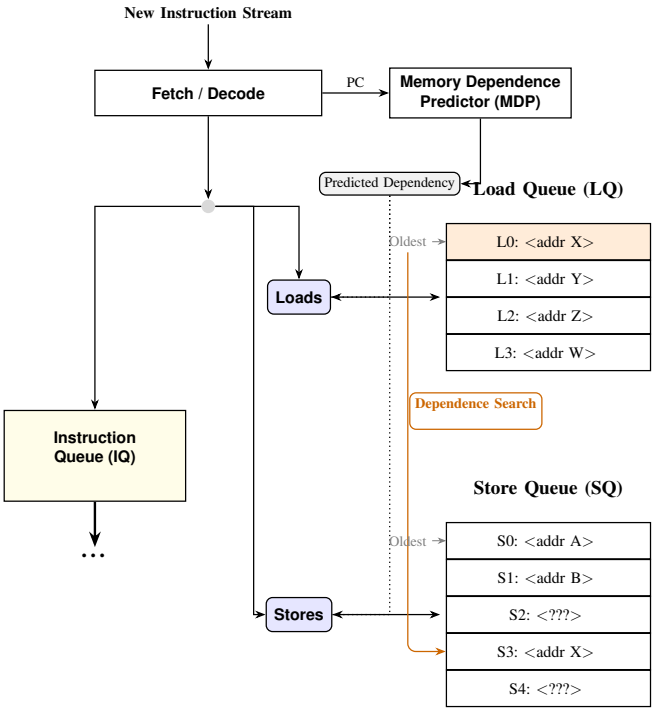


Fig. 2: Diagram of components used to schedule memory operations in out-of-order cores. Memory operations are inserted into the instruction queue according to register dependencies and any predicted memory dependencies, and inserted into the LSQ by program order. When loads execute they search the SQ for forwarding opportunities. When stores execute they search the LQ for memory order violations.

is the MDP algorithm implemented in the open source gem5 simulator [**?**]. Store Sets works by using Store Set IDs (SSID) to track dependent load-store pairs, assigning each instruction the same ID value in the PC-indexed Store Set ID Table (SSIT). The SSIDs are then used to index the Last-Fetched Store Table (LFST), which holds the sequence number of the most recently issued store with an SSID mapping to that LFST entry. Newly fetched loads then access the LFST via their SSID to find the store they're predicted to be dependent on. If a load PC does not map to a valid SSID entry, it is predicted to be memory independent. To forget old dependencies and reduce table pressure, the SSIT and LFST are cyclically cleared after a certain number of fetched memory operations.

Store Sets is extremely area efficient, delivering a large portion of available performance with a tiny hardware budget. Its small size and simplicity make it well suited to area-constrained processors. However, especially at smaller sizes, Store Sets struggles with false dependencies due to hash collisions. When a memory independent load hashes to an existing SSID entry, it is incorrectly made dependent on the corresponding store for that entry. The clear period can be made shorter to offset this, but this comes at the cost of a higher rate of memory order violations as the predictor must re-learn true dependencies more often. Furthermore, many loads exhibit non-consistent memory dependencies. A load in a loop may only be dependent on a store every other iteration, and as Store Sets has no way to disambiguate these cases it conservatively predicts the load as dependent in every iteration.

C. XiangShan Store Sets

XiangShan is an open-source high-performance RISC-V processor RTL implementation [**?**]. XiangShan represents the cutting edge in open-source superscalar processor design, and comes with a gem5 fork modified to more closely model the RTL implementation [**?**]. This gem5 fork implements a variation of the Store Sets predictor that offers a closer representation to implementations found in real processors.

The main optimization over original Store Sets is using multiple 'slots' for each LFST entry, thereby recording multiple dependent stores at once. This allows a load to track multiple dependencies with only one SSID. This is useful in situations where a load is dependent on different stores in different program contexts, or may have partial address overlap with multiple stores. There are further small optimizations to allocation policy which we omit here.

III. Method

This section presents the profile-guided memory dependence prediction (PG-MDP) technique. We put forward the concepts of store distances, the categories of load behaviors PG-MDP labels, and the profiling algorithm to find these loads. We then motivate using profiles, and propose how load labels could be encoded in major ISAs today.

A. Store Distances & Target Load Behaviors

This paper uses store distance to refer to the number of stores in program order between a load and its dependent store inclusive. A load depending on the most recent prior store has a store distance of 1. This concept readily maps onto processor architecture because, as overviewed in Section II-A, stores in the store queue (SQ) are also inserted in program order, so a load's store distance in program order is synonymous with the number of prior SQ entries between the load and the dependent store. Formally, let x be the address of a load, and let the prior stores in program order have addresses $y_1, y_2, \ldots, y_n$ ordered from youngest (y_1) to oldest (y_n). The *store distance* is defined as the minimum index i such that $x = y_i$. If no such index exists, the store distance is ∞.

The premise of PG-MDP is that loads with store distance longer than a certain threshold in 95% or more executions are good candidates to be labeled. The optimal threshold to filter loads by is tightly tied to the hardware parameters of the target processor, so a single optimal threshold can be selected on a per-processor basis in tandem with the overall design.

The heuristic of filtering for sufficiently long store distances captures four different types of behaviors:

- **(1) Memory independent loads:** Loads which read constant data (i.e. have infinite store distance).

- **(2) Rarely dependent loads:** Loads which observe short dependent store distances in <5% of executions.
- **(3) Structurally independent loads:** Loads with no dependent stores in-flight at the time the load is issued.
- **(4) Fast-to-Resolve Dependencies:** Loads which have in-flight yet resolved dependent stores, allowing forwarding.

Loads with behavior type (1) are always labeled regardless of the threshold value. Whether a load exhibits behaviors (2) to (4) depends on the target processor. For instance, a processor with a longer SQ will hold more in-flight stores at once, so only loads with longer store distances will be labeled. The proportion of loads that exhibit type (4) behavior is even further target specific, but is still correlated with a longer store distance as this puts more time between dispatching the store and issuing the load, making it more likely the store's address will be resolved when needed.

B. Profiler Implementation & Overhead

For total precision this paper uses binary instrumentation on train inputs to derive store distances. The core of store distance computation is load and store memory address comparison, widely used in existing techniques such as address sanitizers [**?**]. The restricted scope of PG-MDP permits a simpler algorithm still. The memory address of each executed store is recorded in a FIFO queue of the same capacity as the store distance threshold of the target processor (e.g. 8). When executing a load the queue is searched to find the first prior occurrence of a matching address, if any. Each static load holds an entry in a PC-indexed hash map recording both the number of times a match did and did not occur. After program completion the hash map of load instructions is filtered for loads that did not find matches in the queue in at least 95% of executions. The resulting list is then the load PCs to be labeled. For ease of implementation, load labels in this work are implemented in simulation using a text file of load addresses. In real deployment this list would be used to patch the program binary at the specified addresses with alternate opcodes. As instruction bandwidth is unchanged this would yield identical results.

As the compute complexity is constant for each load and store (with storage growing linearly with static loads), overhead comes almost entirely from per-instruction instrumentation itself, which could incur 10-50x slowdowns using tools such as DynamoRIO [**?**]. Current trends suggest this may be tolerable, with recent work on branch prediction hardware-software co-design [**?**] proposing significantly greater overhead to chase similar performance gains. Nonetheless we will now explore possible alternatives that trade precision for overhead.

Compile-time instrumentation, used by techniques such as address sanitizers, can profile store distances at less than a 2x slowdown [**?**], but is blind to stores occurring in library code. This can be mitigated by function metadata already employed by compilers to convey whether a given callsite may store to a given memory location (e.g. whether the call only touches memory passed through arguments), but beyond commonly used libraries such as libc this metadata is often incomplete and conservative (e.g. the function may store anywhere), which may limit the number of loads that can be confidently labeled.

Full binary instrumentation could be relaxed by only activating per-instruction granularity in code regions visited a certain number of times, similar to how JIT compilers determine what code to optimize. Effective hot code selection would have little to no impact on resulting performance as only cold loads would no longer be labeled. This could further be combined with statistical sampling of store distances rather than testing every execution of a load, again limiting how often per-instruction granularity is used. We aim to prototype and evaluate such techniques in future work.

C. Motivating Profiles

While profile-guided optimization (PGO) has long offered performance benefits, the technique has historically struggled to achieve adoption due to complicating the compilation workflow [**?**]. PGO also introduces concerns about the accuracy of generated profiles, and the potential to harm performance should real input differ too much from the train input.

However, prior work shows that use of PGO has risen significantly in recent years [**?**]. This can be seen in enterprise projects such as Firefox and Chrome web browsers, the Python interpreter and the GCC compiler. Performance-critical server workloads have also seen renewed attention to the benefits that PGO can provide [**?**]. This supports the use of profiles as a reasonable approach to designing new hardware-software co-designs.

To show that PG-MDP generalizes, we compare which loads are labeled by profiles generated from both train and reference inputs for intspeed, shown in Figure 3. 'Positive' means the load is labeled and 'negative' means it is not, and so true positives are loads which are labeled in both profiles, false positives are loads only labeled in the train profile, and likewise for negatives. 'Missing' means the load does not appear in the train workload. These results suggest that profiles generated from train inputs do generalize effectively to reference inputs, with the majority of loads as either true positives (TP) or true negatives (TN). Some missed potential is seen with false negatives (FN), but minimal harmful labels are introduced as false positives (FP). It is also important to note that in the case of load instructions which are unseen in the train inputs, these are never labeled and hence execute as normal. This allows PG-MDP to be more tolerant to workloads with high code coverage variation between inputs.

Lastly, profiles are able to provide further benefits than static analysis. Of the four types of load behavior referred to in Section III-A, only two (types (1) and (3)) could theoretically be captured by perfect alias analysis. Type (2) would further require perfect memory dependence analysis, and there exists no analysis in conventional compiler design that attempts to capture the behavior in type (4). It is also important to note that the alias and dependence analysis offered by industrial compilers today is far from perfect [**?**]. While stronger analyses

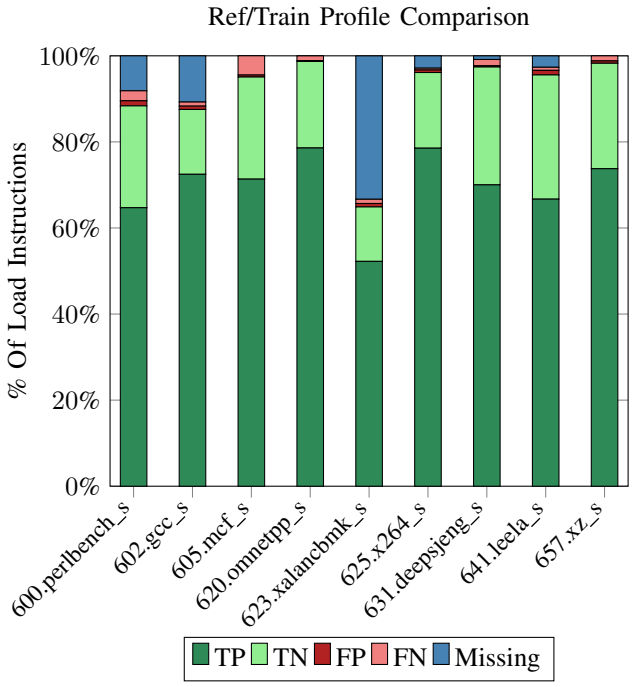


Fig. 3: Comparison of labeled (static) loads between profiles generated from intspeed train and ref inputs with a store distance threshold of 8. Most labels are true positives/negatives when compared to the reference inputs, and very few are false positives.

do exist [**?**], [**?**], these are either only effective in domain-specific languages or do not scale to large programs, making them either less applicable or less feasible. In contrast, profiles allow much greater flexibility and coverage of load behavior, which pairs well with speculative execution where mistakes will not invalidate program semantics.

D. Encoding Labels

PG-MDP requires only a binary ISA annotation that distinguishes conventional loads from labeled loads, and so comes with no increase in instruction bandwidth or change to binary layout. This is practically implementable in both RISC-V and AArch64, which provides coverage for the majority of efficiency-focused cores deployed today. RISC-V reserves four major custom opcodes for target-specific use [**?**], and so standard $LOAD$ instructions could be mirrored to just one of these while retaining the ordinary I-type layout and other ISA semantics. In AArch64, loads are distinguished by a two-bit opc field, and $opc = 1x$ is left unallocated as this would encode a meaningless sign-extension [**?**, §C4-294 to C4-296]. This leaves open an additional bit that could be used to distinguish labeled and non-labeled loads, while remaining meaningless on other targets. In both cases the ISA is able to be modified in a way where targets that benefit from PG-MDP can use labeled loads, and targets that don't see no difference. For other ISAs such as x86 where finding encoding space may not be possible, [**?**] proposes a bitmask in the program binary coupled with pre-decode logic as instructions enter the i-cache. This will incur overhead but would still maintain the critical requirement of not increasing instruction bandwidth.

IV. Evaluation

This section presents the experimental design and results of applying PG-MDP to SPECspeed 2017. We first demonstrate the extent to which the predictor working set can be reduced. We then present a breakdown of per-workload performance gains on the small core, and show how PG-MDP performs on a medium sized core scaled to double the size of the small core. Lastly we evaluate the minimum number of MDP read ports required with and without PG-MDP to achieve maximum performance.

A. Experimental Design

We use an optimized gem5 fork [**?**] that includes an implementation of the XS Store Sets predictor to model the small and medium cores. The full parameters of each model are detailed in Table I. We evaluate each AArch64-compiled SPECspeed 2017 workload using up to 10 simpoints [**?**], with an interval of 100M instructions and an additional warm-up of 10M instructions. Results show the weighted average of the intervals. Some benchmarks run with different inputs, represented with an input number at the end of their label. gem5 is modified to bypass MDP queries for labeled loads and to avoid creating new entries in the case of memory order violation. Violating labeled loads still roll back as normal and do not alter program semantics.

The modeled small core is intended to resemble out-of-order efficiency cores deployed today, such as Apple's M-series efficiency cores. We stress that this correspondence is only approximate, as gem5 is a model with its own micro-architectural assumptions and features. The design class is modeled with a configuration that is appropriate to gem5 rather than by replicating any single processor. We configure XS Store Sets to use 128 SSIT entries and 64 LFST entries with two slots each, coming to a baseline-favourable 300B of MDP storage.

The medium core is used to explore how PG-MDP's impact changes as MDP size and available ILP increase, each of which is doubled over the small core. However as previously discussed this use case is somewhat less realistic as larger cores are more likely to employ additional hardware techniques which influence the baseline predictor working set.

B. Performance

Figure 4 shows the percent IPC changes in each SPECspeed workload for the small core configuration. It can be seen that IPC gains have an uneven spread across workloads, with some seeing large benefits (up to 25%) and others next to none. This correlates with the magnitude of false dependencies before and after applying PG-MDP seen in Figure 5. The largest gainers such as $607.cactuBSSN_s$ and $625.x264_s$ all exhibit relatively higher false dependencies per kilo-instruction in the base

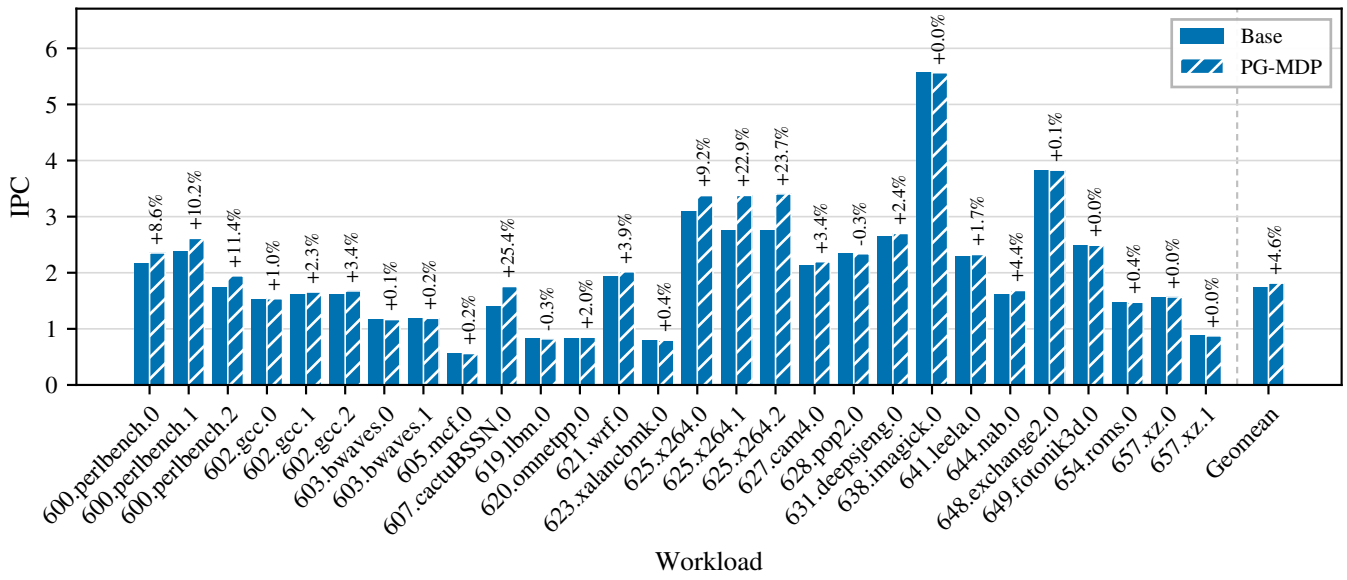


Fig. 4: IPC % improvement with PG-MDP for each workload on the small core configuration. A handful of workloads benefit significantly, whereas others are largely unchanged. Annotations omit changes $< 0.5\%$.

TABLE I: Parameters of processor components for each simulated model

		Small	Med.
Instruction Window	ROB:	128	256
	IQ:	77	154
	LQ:	41	85
	SQ:	26	66
Pipeline Width	Fetch/	6	8
	Commit:	8	12
	Issue:		
XS Store Sets	SSIT:	128	256
	LFST:	64	128
	Slots:	2	2
	Clear:	125k	125k
L1D	Size:	32KB	64KB
	MSHRs:	16	32
	Prefetcher:	Stride	
L1I	Size:	32KB	32KB
	MSHRs:	16	32
	Prefetcher:	Tagged	
L2	Size:	1MB	2MB
	MSHRs:	32	64
	Prefetcher:	Stride	
L3	Size:	2MB	4MB
	MSHRs:	64	64
	Prefetcher:	Stride	
Branch Predictor:		64KB TAGE-SC-L	
Store Distance Threshold:		8	22

case, and see this cut down significantly by over 80% with PG-MDP. Figure 7 further shows the reduction in dynamic MDP load queries per kilo-instruction, demonstrating the impact on predictor working set. Figure **??** shows the change in memory order violations.

Listing 1 uses an example function from $625.x264_s$ to demonstrate why PG-MDP is impactful. The code is a simplified version of $pixel_avg$, a hot function that causes a large portion of false dependencies. The loads on $src1$ and

Listing 1: Example function from 625.x264_s

```
void pixel_avg( uint8_t *dst, uint8_t *src1,
    uint8_t *src2,
                int i_width, int i_height )
{
    for ( int y = 0 .. i_height )
    {
        for( int x = 0 .. i_width )
            dst[x] = ( src1[x] + src2[x] + 1 )
                >> 1;
    }
}
```

$src2$ never observe memory dependencies during program execution. As the loop is both short in instructions and part of a tight nest, these loads easily fall on the critical path during execution. Due to hash collisions, the loads are falsely made dependent on unrelated stores and the loop is unnecessarily serialized. Furthermore, the loop is compiled into three distinct variants that load different data sizes depending on remaining pixels, increasing both static load and store count pressure and making hash collisions more likely. Using PG-MDP, loads from all iterations across all access sizes are free to issue in parallel, significantly improving ILP. Altogether this represents a perfect use case for PG-MDP and is a significant contributor to the resulting performance gain.

PG-MDP is more effective on small predictors where aliasing happens more frequently. To demonstrate this we performed the following analysis presented in Figure 6. We vary the size of the MDP (x axis) and show how performance differs with and without PG-MDP, when evaluated on the medium core. As expected, the greater available ILP increases PG-MDP's impact on performance. However this is out-competed

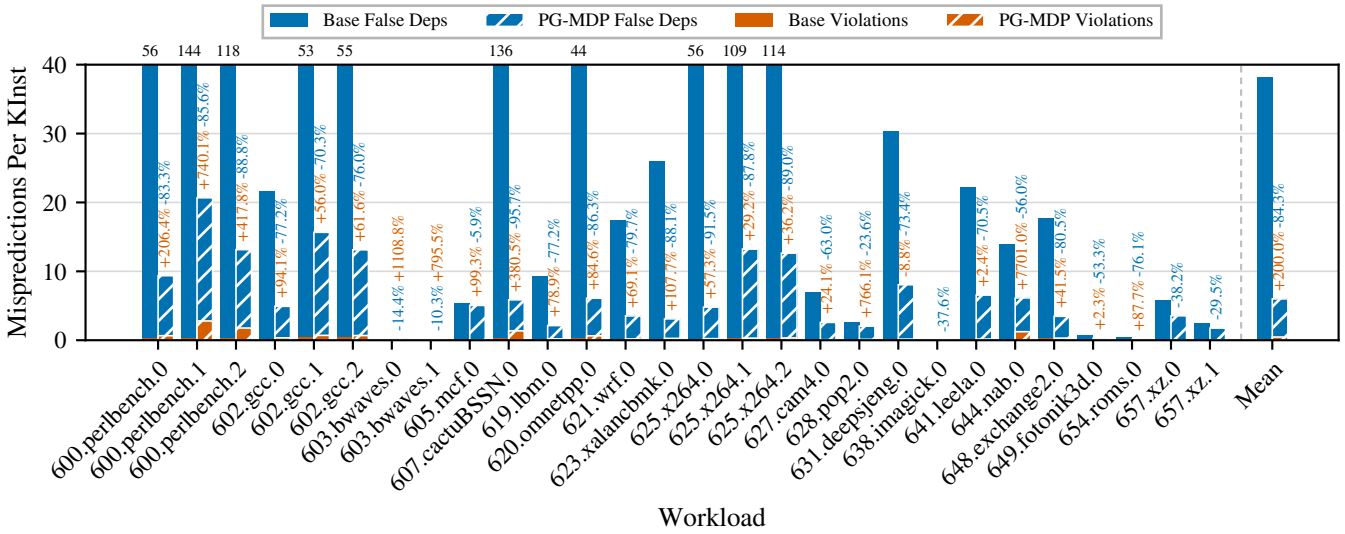


Fig. 5: Percent change of false dependencies per kilo-instruction. PG-MDP benefits workloads most that by default have a higher than average rate of false dependencies, but is still able to cut false dependencies across all workloads.

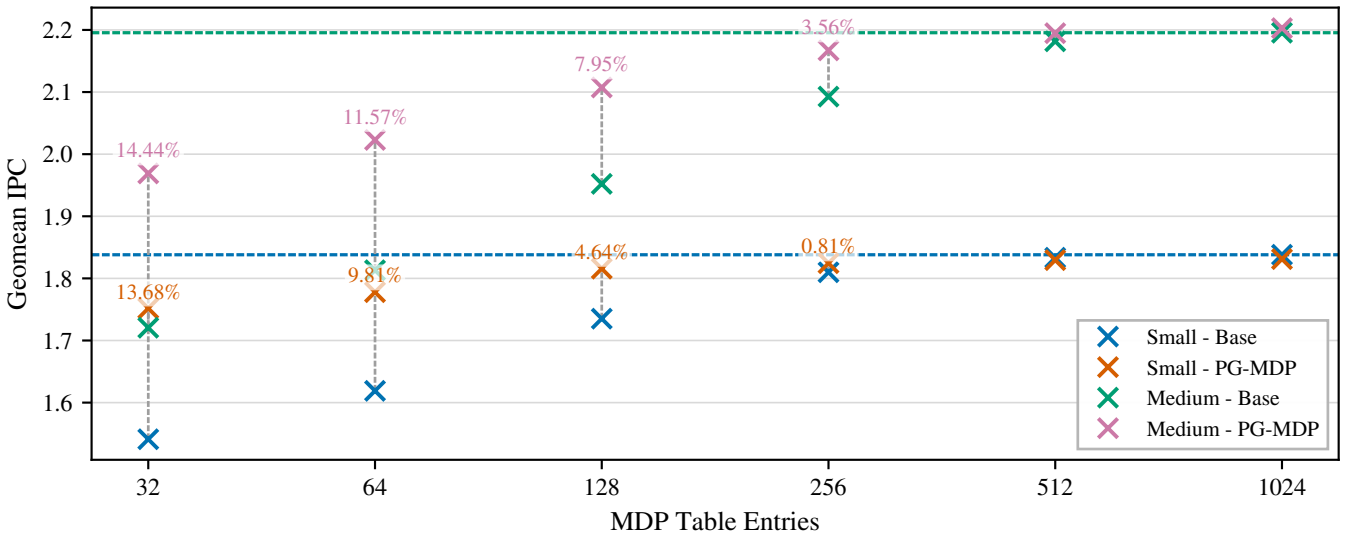


Fig. 6: Base and improved IPC across SPECspeed 2017 for increasing XS Store Sets [**?**] sizes on a medium core (ROB=256). IPC gains increase over the small core, but for the baseline MDP size (256 entries), IPC improvement is still 1% smaller.

by using a larger MDP which diminishes opportunities for PG-MDP to reduce false dependencies, reducing the total IPC gain from 4.6% to 3.6%. As such while PG-MDP is still able to deliver gains to larger cores, these results confirm that the technique is best suited to smaller cores that cannot afford larger MDPs.

C. Reducing MDP Read Ports

Because PG-MDP reduces the MDP working set so significantly, it could permit designing MDPs with fewer read ports. To test this, we extend gem5 to model MDP read ports and evaluate how many ports are required to achieve maximum performance with and without PG-MDP. This is implemented by constraining the maximum number of both loads and stores which can be dispatched in a cycle before the pipeline stage blocks. We optimistically assume MDP queries always return within the current cycle, meaning this counter resets to zero every cycle. For example, 8 read ports mean any combination of 8 load or store instructions may be dispatched in a cycle, and encountering more will stall dispatch until the next cycle. Loads labeled by PG-MDP do not occupy a read port and so do not increment the counter.

Figure 8 shows the geomean IPC of the small core for the XS Store Sets predictor with and without PG-MDP. We vary the number of read ports of the MDP (x axis). In the baseline, 5 read ports are required for maximum performance, whereas PG-MDP sees higher IPC with only 3 ports, and comes within 1% of the maximum baseline performance with only 2 ports.

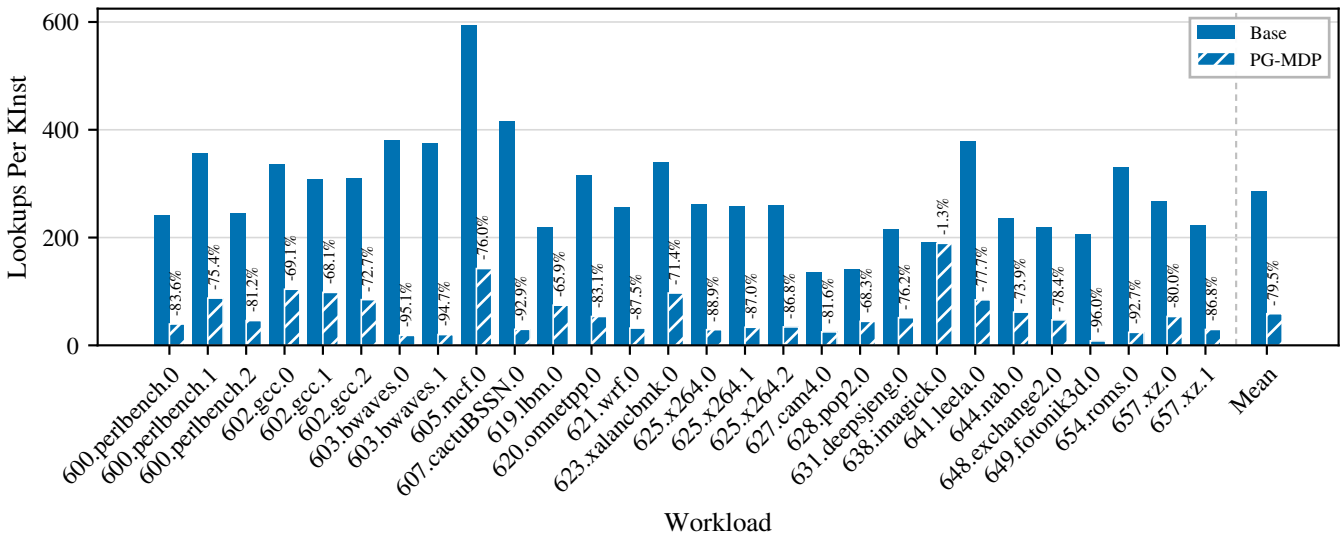


Fig. 7: Percent change of MDP load queries per kilo-instruction. Demonstrates the extent to which PG-MDP reduces the MDP working set by reducing how often the MDP is queried for predictions.

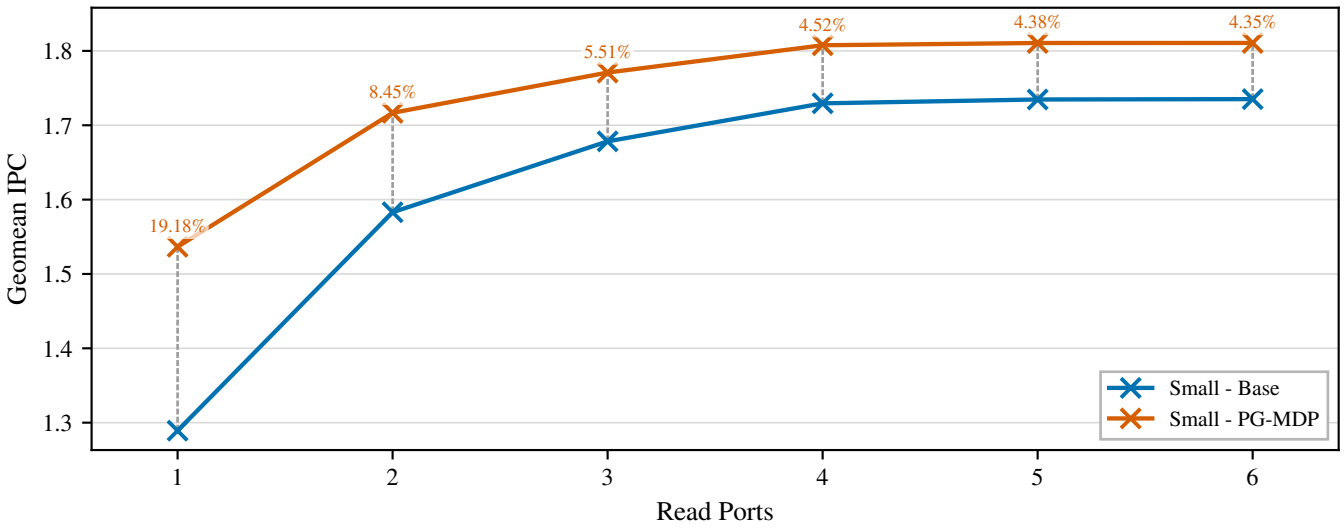


Fig. 8: Geomean IPC of the small core across SPECspeed 2017 against the number of MDP read ports, with and without PG-MDP. Using PG-MDP requires three fewer ports to achieve within 1% of using five in the baseline.

This suggests shrinking the working set can help overcome this design constraint in MDP, and could also make growing the predictor more feasible.

This evaluation should not be confused with a claim of how many MDP read ports real processors require. We do not evaluate predictor latency, nor the low level techniques that may or may not hide that latency, and so these results must be understood in a relative sense, i.e. through the comparison between baseline and PG-MDP, instead of as an absolute claim.

D. GAPBS

E. MDP Comparison

V. Related Work

Improving Memory Dependence Prediction with Static Analysis [**?**] tackles the same problem as this paper, but by using static analysis to determine which load instructions to label. It demonstrates small (<1%) IPC gains in select workloads, but with an overall best-case geomean improvement of less than 0.1%. This is due to only reducing average false dependencies by 7.5%, over 10x less than PG-MDP. Furthermore, due to relying on static analysis it also sees notable performance regressions elsewhere, hurting viability. In comparison, by leveraging profiles we are able to achieve much higher IPC gains while also avoiding significant regressions.

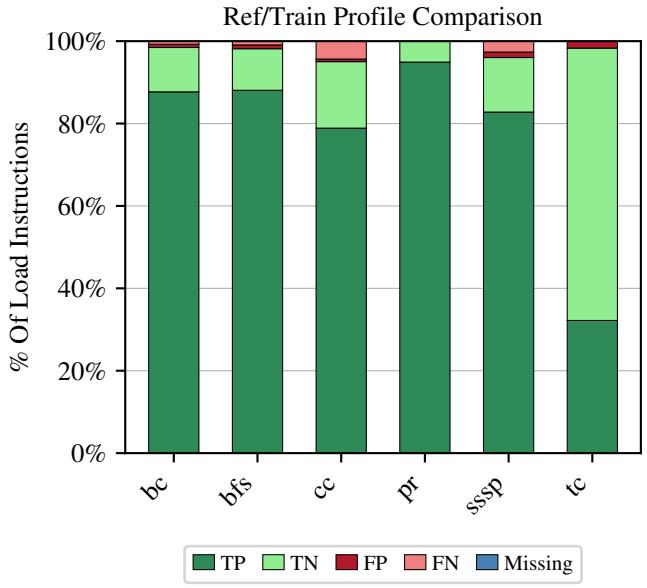


Fig. 9

Feedback-directed Memory Disambiguation Through Store Distance Analysis [**?**] proposes a similar profile-guided co-design to this paper, but with a more aggressive use case. In their work, the MDP is replaced altogether and load instructions are extended to include 4-bit distance fields indicating the store queue position they're likely to be dependent on. This achieves high accuracy compared to Store Sets, but comes at the cost of higher instruction bandwidth to encode predicted store distances, which is usually infeasible in modern processor designs where instruction bandwidth is critical. Larger processors would also require more bits to encode longer store queue distances. Furthermore, [**?**] is more sensitive to profile accuracy than our work as loads which do not appear in the profile must be assumed to be memory independent, likely leading to a spike in memory order violations in workloads like $623.xalancbmk_s$ with low common code coverage between train and ref inputs. In contrast, our technique is able to leave unseen loads to execute as normal. Attempting to replace the MDP altogether also hurts deployment feasibility, as it requires all binaries be compiled with this profile-guided technique, whereas our technique only supplements the MDP and so allows unlabeled binaries to still execute at baseline performance.

Software-hardware Cooperative Memory Disambiguation [**?**] is a binary analysis co-design to reduce load queue pressure. Certain kinds of provably memory independent loads are labeled and prevented from being inserted into the load queue when issued. This technique is able to label 40% of static loads across SPEC2006 floating point (but dynamic percentage is unclear). This also incurs additional instruction bandwidth by inserting barrier-like instructions when entering loops to ensure labeled loads are only removed from the load queue when their parent loop reaches a steady state in the pipeline. LSQ entries must also store an additional bit to track whether these custom instructions are in-flight or not. This work presents an interesting proposal for reducing load queue pressure in area-constrained processors that lack space for hardware-based load elimination techniques, but has reduced viability due to lacking a mechanism for ensuring memory coherence in multi-core systems.

Constable: Improving Performance and Power Efficiency by Safely Eliminating Load Instruction Execution [**?**] is a pure hardware technique to track and eliminate stable loads which consistently read the same value from memory, i.e. behavior type (1) specified in Section III-A. This technique is able to eliminate both the memory read and address calculation, greatly improving pipeline efficiency. It also demonstrates a broad coverage across a variety of workloads, improving geomean IPC by 5.1% on a high-performance core. For high-performance cores this technique would be effective for reducing the MDP working set, however as discussed in Section I, expensive hardware techniques are less viable on area-constrained cores due to area & power demands, as well as requiring abundant ILP to effectively return on investment.

Load-Wait Tables A load-wait table is a very simple type of memory dependence predictor found in the Alpha 21264 [**?**]. It consists of a PC-indexed bitvector which tracks whether a given load has ever triggered a memory order violation, stalling it on all prior stores on hit and issuing as soon as possible on miss. This aims to answer the same question in hardware that PG-MDP does in software, i.e. is a given load memory independent. One might imagine a combined approach of filtering the working set of a more sophisticated predictor with a small load-wait table. We argue that PG-MDP still compares favorably against such a scheme; a load-wait table would still be queried by all loads, and so inherits the same multi-porting constraint discussed in Section I. It would then also suffer from table saturation at any reasonable hardware budget, and so would need to be periodically cleared and re-trained to adapt to program phase changes. This would cause spikes in false dependencies during saturation and spikes in memory order violations after clearing, limiting the accuracy it can achieve despite still incurring additional hardware costs.

Other Memory Dependence Prediction Algorithms Besides Store Sets and load-wait tables [**?**], [**?**], other MDP algorithms include Store Vectors, MDP-TAGE, and PHAST/MASCOT [**?**], [**?**], [**?**], [**?**], [**?**] (MASCOT is an extension of PHAST, but with a focus on speculative-memory bypassing and performs similarly for strictly MDP). Each of these algorithms offer different trade-offs and advantages. PHAST/MASCOT is the current state of the art, but uses >10x the storage current high-performance cores are estimated to allocate to MDP. We choose to evaluate a Store Sets-based predictor because it achieves the highest performance of feasible predictors in the literature for small cores [**?**]. We chose XS Store Sets [**?**] specifically as it is, to the best of our knowledge, the most modern implementation of Store Sets and closest available representation of MDP algorithms found in real processors.

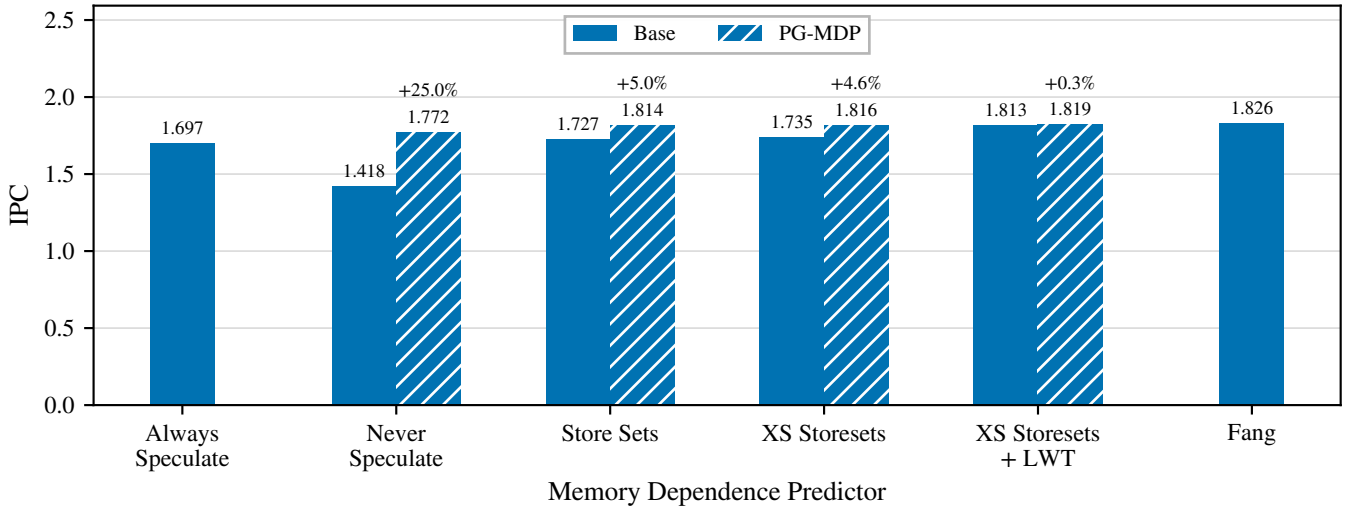


Fig. 10

VI. Future Work

Per-Workload Threshold Selection could achieve higher performance than per-core thresholds. As the threshold is untied to any architectural state, it is possible to select a different optimal store distance threshold for each individual workload. We chose not to do this in this work to prove that meaningful performance gains were possible with a generic threshold. However, there are use cases where this is feasible and it would be worthwhile to evaluate the potential performance available for the additional offline cost.

Just-in-Time Compilers (JITs) are a widely used run-time based compilation technique for dynamic languages. PG-MDP could pair well with these compilers for various reasons. Firstly, the additional workflow complexity of profiling can be automated by the run-time optimizer. Secondly, JITs make speculative optimizations about observed program behavior, and so could aggressively apply ISA labels to any loads which read from addresses not recently stored to. Lastly, dynamic languages tend to be implemented with many memory independent pointer-chasing loads. This suggests these workloads could have a high potential for performance gains from using PG-MDP.

Serverless Workloads are characterized as a collection of many short-lived workloads which only execute a few times, then not again for a long time with lots of unrelated code in-between. This effectively makes their execution always cold on the target processor, so high performance is difficult to achieve. When memory dependence predictors cannot learn dependencies fast enough to deliver performance gains, it may be preferable to disable memory dependence prediction entirely and conservatively stall loads on any unresolved stores. In this case, PG-MDP could potentially deliver substantial IPC gains, even on large cores, by allowing likely-independent loads to still issue immediately.

VII. Conclusion

This paper proposed profile-guided memory dependence prediction (PG-MDP), a profile-guided co-design to label load instructions via their opcode and prevent them from making MDP queries. This was shown to reduce the predictor working set and reduce false dependencies by 84%, yielding a 4.6% IPC gain on an area-constrained core, achieving performance within 1.2% of using 8x more predictor entries. These results point to a promising new direction for improving IPC on area-constrained, efficiency-focused cores; by attacking predictor working set size in software, the penalties of small capacity hardware structures can be almost entirely mitigated while staying within the strict design constraints of these cores.